

PiPER Robot Arm: WSL2 and Python Installation Guide

Purpose. This guide records the installation and verification process used to control an AgileX PiPER 6-DOF robot arm from Python on a Windows computer, using Ubuntu under WSL2 and a USB-CAN adapter. The goal is to use pyAgxArm directly, without ROS or ROS2.

Estimated total installation time: approximately 30–90 minutes of hands-on setup, plus potentially longer kernel build/download time depending on computer performance and internet speed. This is an estimate, not a measured duration.

High-level installation sequence

1. Prepare Windows and WSL2, and install/launch Ubuntu.
2. Build a custom WSL Linux kernel with CAN and gs_usb USB-CAN support.
3. Configure WSL to use the custom kernel and restart WSL.
4. Attach the USB-CAN adapter to WSL and bring up can0 at 1 Mbps.
5. Create a Python virtual environment and install python-can and pyAgxArm.
6. Run a connection test, read arm status and joint angles, and verify repeated feedback.

Important safety note. The verification commands in this document are read-only. Do not run motion commands or uncomment movement examples unless you have reviewed the command, cleared the workspace, secured the arm, removed any load from the gripper, and can reach the emergency stop. Warn the operator before any test that may move the arm, including homing.

Command-by-command record

Run commands in the order shown. Commands marked “(example)” are templates and may need their paths or values adjusted for the computer being configured. The Windows PowerShell commands that use usbipd should be run in an Administrator PowerShell window where noted.

Step / command	Terminal	What it does	Expected time
Open PowerShell as Administrator	Windows PowerShell (Admin)	Allows USB/IP binding and attachment commands.	fast
wsl --list --verbose	Windows PowerShell	Checks installed WSL distributions and whether Ubuntu uses WSL 2.	fast
wsl --install -d Ubuntu	Windows PowerShell (Admin)	Installs Ubuntu under WSL if it is not already installed; skip if already installed.	Several minutes
wsl --shutdown	Windows PowerShell	Stops WSL so kernel configuration changes can take effect.	fast
git clone --depth=1 -b linux-msft-wsl-5.15.167.4 https://github.com/microsoft/WSL2-Linux-Kernel.git	Ubuntu (WSL)	Downloads the matching Microsoft WSL kernel source branch.	1–5 minutes
cd ~/WSL2-Linux-Kernel	Ubuntu (WSL)	Moves into the kernel source directory.	fast
zcat /proc/config.gz > .config	Ubuntu (WSL)	Uses the currently running kernel configuration as the starting point, if /proc/config.gz is available.	fast
make menuconfig	Ubuntu (WSL)	Opens kernel configuration; enable CAN support, CAN_RAW, CAN_DEV, and CAN_GS_USB. Save the configuration.	5–20 minutes
make prepare modules_prepare	Ubuntu (WSL)	Prepares kernel source and headers for building.	1–5 minutes
make -j\$(nproc) modules	Ubuntu (WSL)	Builds kernel modules, including the USB-CAN driver.	5–30 minutes
sudo make modules_install	Ubuntu (WSL)	Installs the built modules into the WSL kernel module directory.	1–5 minutes
make -j\$(nproc)	Ubuntu (WSL)	Builds the custom Linux kernel image.	5–30 minutes
cp arch/x86/boot/bzImage /mnt/c/Users/<WindowsUser>/wsl_kernel	Ubuntu (WSL)	Copies the built kernel image to a Windows path accessible to WSL configuration. Replace	fast

		<WindowsUser>.	
Create %UserProfile%\wslconfig with kernel=C:\\Users\\<WindowsUser>\\wsl_kernel	Windows (edit file)	Tells WSL to boot the custom kernel. Use the exact Windows path and the [wsl2] section.	fast
wsl --shutdown	Windows PowerShell	Restarts WSL cleanly so it loads the configured kernel.	fast
uname -r	Ubuntu (WSL)	Verifies that the custom kernel is running; the tested setup showed a version ending in "+".	fast
usbipd list	Windows PowerShell	Finds the USB-CAN adapter and its current BUSID; BUSID can change after reconnecting.	fast
usbipd bind --busid <BUSID>	Windows PowerShell (Admin)	Binds the adapter for USB/IP sharing. Usually needed once per device; replace <BUSID>.	fast
usbipd attach --wsl --busid <BUSID>	Windows PowerShell (Admin)	Attaches the adapter to WSL. Repeat after detach/reconnect or a WSL restart if necessary.	fast
lsusb	Ubuntu (WSL)	Verifies that the USB-CAN adapter is visible inside Ubuntu.	fast
sudo modprobe gs_usb	Ubuntu (WSL)	Loads the candleLight/gs_usb CAN driver and should create can0 when the adapter is attached.	fast
ip link show can0	Ubuntu (WSL)	Checks whether the CAN network interface exists.	fast
sudo ip link set can0 up type can bitrate 1000000	Ubuntu (WSL)	Brings up can0 at 1 Mbps, matching the tested configuration. Confirm the required bitrate for your robot/adapter.	fast
ip -details link show can0	Ubuntu (WSL)	Verifies that can0 is UP and reports CAN link details.	fast
python3 -m venv ~/piper-env	Ubuntu (WSL)	Creates an isolated Python environment.	fast
source ~/piper-env/bin/activate	Ubuntu (WSL)	Activates the environment; the prompt should show (piper-env).	fast
python -m pip install --upgrade pip	Ubuntu (WSL), venv active	Updates pip in the environment.	Fast to 1 minute
python -m pip install python-can	Ubuntu (WSL), venv active	Installs Python CAN support.	Fast to 1 minute
python -m pip install "git+https://github.com/agilexrobotics/pyAgxArm.git"	Ubuntu (WSL), venv active	Installs pyAgxArm from the AgileX GitHub repository.	1–3 minutes
python -c "import pyAgxArm; print('pyAgxArm imported successfully')"	Ubuntu (WSL), venv active	Checks that pyAgxArm imports.	fast
python -c "from pyAgxArm import create_agx_arm_config, AgxArmFactory, ArmModel, PiperFW; r=AgxArmFactory.create_arm(create_agx_arm_config(robot=ArmModel.PIPER, firmware_version=PiperFW.DEFAULT, interface='socketcan', channel='can0')); r.connect(); print(r.get_arm_status()); print(r.get_joint_angles())"	Ubuntu (WSL), venv active	Connects to the PIPER and reads status and joint angles. This is a read-only test.	fast
python -c "from pyAgxArm import create_agx_arm_config, AgxArmFactory, ArmModel, PiperFW; import time; r=AgxArmFactory.create_arm(create_agx_arm_config(robot=ArmModel.PIPER, firmware_version=PiperFW.DEFAULT, interface='socketcan', channel='can0')); r.connect(); exec('while True:\n print(r.get_joint_angles())\n time.sleep(0.5))"	Ubuntu (WSL), venv active	Continuously prints joint-angle feedback every 0.5 seconds; stop with Ctrl+C. Read-only.	Run until Ctrl+C

Verification results from the completed setup

- The custom WSL kernel loaded successfully and the gs_usb driver created can0.
- The can0 interface was brought UP at 1,000,000 bit/s.
- pyAgxArm imported successfully and connected using SocketCAN on can0.
- The arm reported NORMAL status and returned joint angles.
- Repeated joint-angle feedback was received at 0.5-second intervals without issuing motion commands.

INSTALLATION IS COMPLETE. The software installation and read-only communication verification are finished. Actual movement commands and physical motion testing have not been completed.

Prompt for future AI assistance

Use this document as the installation record for setting up an AgileX PiPER 6-DOF robot arm from Windows using WSL2 Ubuntu, a USB-CAN adapter, SocketCAN, and pyAgxArm without ROS or ROS2. Walk me through the process one command at a time. Keep explanations concise, tell me which terminal to use, wait for me to report the result before giving the next command, and adapt if my output differs. Do not assume a USB BUSID or Windows username; help me identify the current values. Distinguish commands that are prerequisites from commands that can be skipped when already completed. Do not run or recommend any command or example that could physically move the robot or gripper without first warning me clearly and getting my confirmation. Prefer read-only checks until I explicitly approve a movement test. At the end, summarize which installation and verification steps succeeded and which remain.